Week 27 Graded Mini Project: Policy Assistant Agent (Single Agent + Memory + Graph)

Arpit Jain | Repo: agentic-policy-assistant/ | Notebook: Week 27_Graded Mini Project_Jain.ipynb | Results: results/*.png

Part 1: Foundations

Task 1.1: Data prep

data/policies.jsonl holds **8 short policies of a fictional company**, one JSON object per line with doc_id, text and date. The doc_ids carry a version (leave_v3), and remote_work_v1 is kept on purpose as a **superseded** version, to test that the agent cites the latest policy.

doc_id	date	words	covers
remote_work_v2	2026-01-15	97	Remote Work Policy
leave_v3	2026-01-01	88	Leave Policy
travel_v4	2026-02-01	87	Travel and Expense Policy
security_v1	2025-11-10	78	Information Security Policy
conduct_v2	2025-09-01	69	Code of Conduct
learning_v1	2025-12-01	59	Learning and Development Policy
performance_v2	2026-03-01	62	Performance and Promotion Policy
remote_work_v1	2024-03-01	39	Remote Work Policy

One JSONL record

```
{"doc_id": "leave_v3", "text": "Leave Policy (v3). Full-time employees receive 24 days of annual
leave ...", "date": "2026-01-01"}
```

Task 1.2: Tools

Tool	Contract	Errors
policy_search(query, k)	FAISS (CPU) over 2-sentence windows of each policy, local MiniLM embeddings; returns JSON hits {doc_id, date, snippet, score}; k is clamped to 1-8	never raises
read_policy(doc_id)	full text + date of one policy	read_policy_text raises the typed PolicyNotFoundError (with close-match suggestions); the tool returns it as a soft 'ERROR ...' string
calculator(expression)	AST-based arithmetic only (+ - * / % **); no names, calls or attributes	CalculatorError for division by zero or non-arithmetic input (soft)

tools.py - typed error, then soft failure in the tool wrapper

```
def read_policy_text(doc_id: str) -> PolicyDoc:
    policies = load_policies()
    doc_id = doc_id.strip().strip("[]")
    if doc_id not in policies:
        suggestions = difflib.get_close_matches(doc_id, list(policies), n=3, cutoff=0.3) or
sorted(policies)[:3]
        raise PolicyNotFoundError(doc_id, suggestions)
    return policies[doc_id]

@tool
def read_policy(doc_id: str) -> str:
    """Return the full text and date of one policy by its exact doc_id (for example leave_v3)."""
    try:
        d = read_policy_text(doc_id)
        return json.dumps({"doc_id": d.doc_id, "date": d.date, "text": d.text})
    except PolicyToolError as e:
        return f"ERROR {e}"
```

Tool outputs

```
read_policy("remote_work_v9.9") -> ERROR PolicyNotFoundError: no policy with doc_id 'remote_work_v9.9'.
Closest known ids: remote_work_v2, remote_work_v1, performance_v2. Use policy_sea...
calculator("12 * 4") -> 48     calculator("5 / 0") -> ERROR CalculatorError: division by zero
```

Task 1.3: Prompts

prompts.py holds the system prompt below, a retry suffix (the critic's issues are fed back to the agent), the memory-hint template and the optional LLM critic prompt. The acceptance rules map one-to-one to the checks the critic and eval.py enforce:

```
You are the Policy Assistant for Kestrel Data Labs. You answer employee questions using ONLY the
internal policy corpus, which you can reach through tools. You have no other knowledge of this company.
```

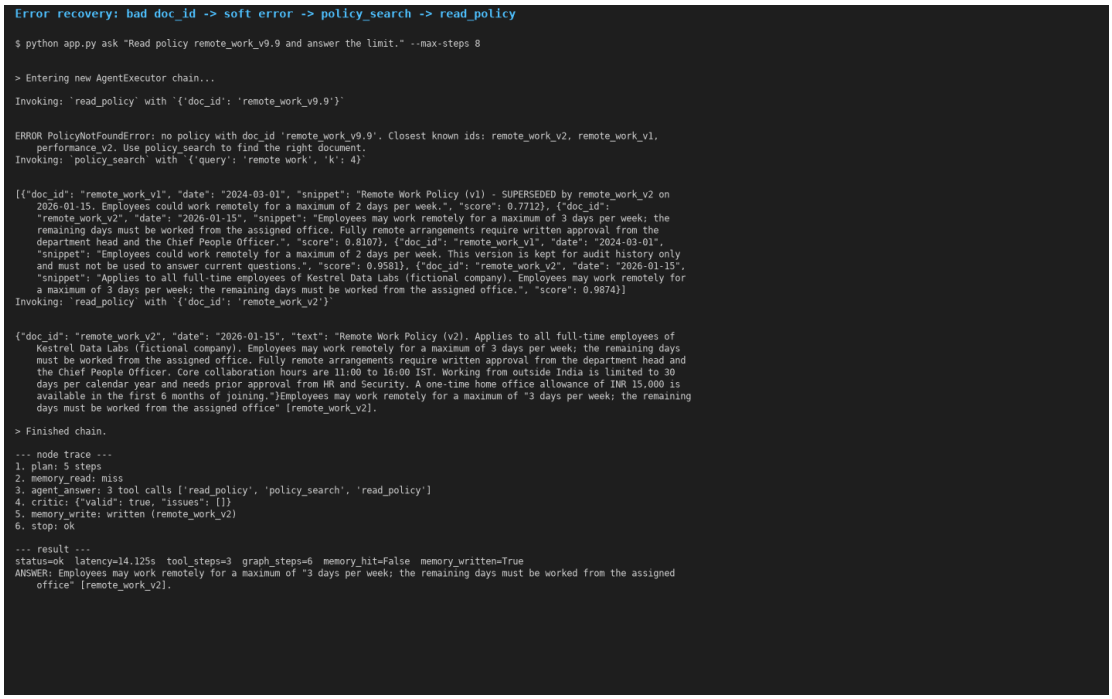
```
TOOLS
- policy_search(query, k): find relevant policies. Start here unless the user gives an exact doc_id.
- read_policy(doc_id): read one full policy. If it returns "ERROR PolicyNotFoundError", do not retry the
same
  id. Tell yourself the id was wrong, call policy_search with the topic instead, then continue.
- calculator(expression): use for any arithmetic. Never do maths in your head.

ACCEPTANCE RULES (every final answer must satisfy all of them)
1. doc_id tokens: cite every policy you rely on as a token in square brackets, e.g. [leave_v3].
   Only cite doc_ids returned by a tool in this conversation.
2. Quotes: for every exact rule, number or limit, include a short verbatim quote (at most 25 words) from
the
   policy text in double quotes, followed by its token, e.g. "Up to 10 unused annual leave days can be
carried
   over" [leave_v3].
3. Grounding: no external facts, assumptions or general HR knowledge. If the corpus does not answer the
   question, say exactly: "I could not find this in the policy corpus." and cite nothing.
4. Versions: if two versions of a policy exist, use the one with the latest date and ignore superseded
ones.
5. Step limit: use at most 6 tool calls. Prefer the cheapest path: if a VERIFIED MEMORY hint
   names a doc_id, confirm it with one read_policy call instead of searching.
6. Format: 1-4 short sentences. If the user asked for a calculation, give its result too.
```

- **doc_id tokens:** rule 1. The critic rejects answers without a valid [doc_id]; eval.py records has_docid.
- **Quoting:** rule 2. The critic checks that every quote appears verbatim in the cited policy.
- **Step limits:** rule 5, enforced in code too: AgentExecutor max_iterations=6 and graph max_steps=8.

Task 1.4: Agent

- **Agent:** create_tool_calling_agent + AgentExecutor (a ReAct loop: the model picks a tool, observes the result, decides again). Model: **gpt-4o-mini**, temperature=0, seed=42, max_iterations=6, verbose=True. Every response is cached in SQLite, so reruns are identical.
- **Observability:** run_agent returns the answer, every tool call (tool, input, output preview), the step count, latency and tokens.
- **Bad-id recovery:** read_policy('remote_work_v9.9') returns a soft PolicyNotFoundError message with suggestions. Prompt rule tells the agent not to retry the same id but to policy_search the topic, then read the latest matching policy. The verbose trace below shows exactly that sequence.



Error-recovery case (terminal output of app.py with the verbose agent trace)

Task 1.5: Evaluation Version 1

8 canned questions: the 6 starter queries + a password-length question + an out-of-corpus boundary question. Pass = has_docid AND has_quote AND expected doc cited AND key facts present (boundary: refuses and cites nothing). **Version 1 = the bare agent (no graph, memory or critic): pass rate 100% (8/8).**

id	kind	latency (s)	steps	has_docid	has_quote	cited	PASS

E1	factual	2.93	1	True	True	remote_work_v2	PASS
E2	factual	2.83	1	True	True	leave_v3	PASS
E3	factual	3.23	2	True	True	travel_v4	PASS
E4	factual	3.34	2	True	True	security_v1	PASS
E5	recovery	4.61	3	True	True	remote_work_v2	PASS
E6	multi-tool	2.82	2	True	True	travel_v4	PASS
E7	factual	2.21	1	True	True	security_v1	PASS
E8	boundary	6.50	4	False	False	-	PASS

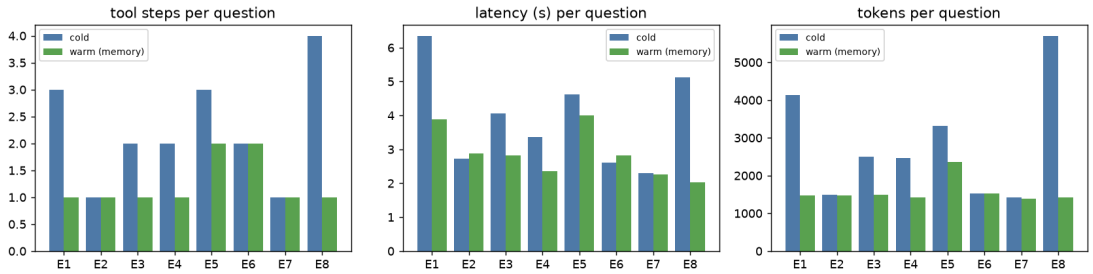
Logs: every question is saved as one JSON line (answer, tool calls, latency, steps) in runs/logs/; the table is runs/eval_v1_agent_only.csv.

Part 2: Memory, Graph, Critic and Polishing

Task 2.1: Memory

- **Write only verified facts:** memory.write stores a fact only if the answer has a [doc_id], every doc_id exists, and every quote is verbatim in the cited policy. The graph calls it only after the critic approves. Duplicates are skipped.
- **Read cheapest sufficient first:** FAISS similarity over stored questions, threshold 0.80 cosine. Among the qualifying facts, the one with the fewest doc_ids (cheapest to re-verify) wins. It is injected into the question as a 'VERIFIED MEMORY' hint, so the agent can confirm with one read_policy instead of searching.

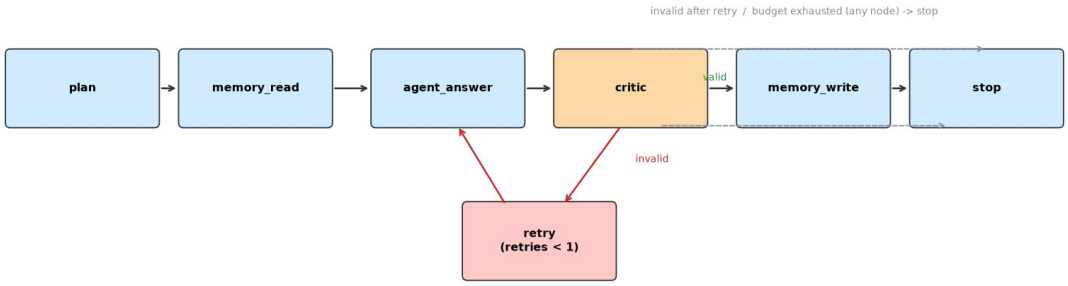
metric	cold	warm
avg latency (s)	3.891	2.882
avg tool steps	2.25	1.25
avg tokens	2821.12	1571.62
pass rate	1	1
memory hits	0	7
memory writes	7	0



Cold run (memory cleared) vs warm run (memory kept), per question

On the warm run, memory hit 7 of 8 questions and cut tool steps by 44%. No new facts were written, because they were already stored. The pass rate stayed at 100%, so the hints made the agent cheaper without reducing grounding.

Task 2.2: Graph orchestration



graph.py - LangGraph StateGraph

- **Nodes:** plan (rule-based tool plan) → memory_read → agent_answer → critic → memory_write → stop. Every node appends a line to state.trace and increments graph_steps.
- **Budget:** a router after every node sends the run to stop with status budget_exhausted once graph_steps reaches max_steps (8). LangGraph's recursion_limit is a hard safety net on top.
- **Single retry:** if the critic rejects the answer, a retry node re-runs agent_answer once with the critic's issues, but only if the budget still fits. A second rejection gives failed_critic, with no memory write.



Budget stop (max_steps=3) and the critic → retry → stop path (scripted bad model)

Task 2.3: Critic

graph.critic_check is deterministic and returns the minimal JSON {"valid": bool, "issues": [...]}. It checks:

- a [doc_id] is present and the doc_id exists; at least one quote; each quote is verbatim in the cited policy
- simple claims: every number in the answer must appear in the cited policies (or come from the question / calculator)
- no superseded policy is cited; a 'not found' answer cites nothing
- optional: an LLM critic (prompts.CRITIC_PROMPT, same JSON) to check claim semantics

Critic verdicts

```
{"valid": false, "issues": ["no [doc_id] token in the answer"]}
    <- Remote work is allowed 3 days a week.
{"valid": false, "issues": ["quote(s) not found verbatim in cited policies: ['Employees may work remotely
every day']"]}
    <- "Employees may work remotely every day" [remote_work_v2]
{"valid": false, "issues": ["numbers not found in cited policies: ['30']"]}
    <- You get 30 days. "Full-time employees receive 24 days of annual leave per calendar year" [leave_v3]
{"valid": false, "issues": ["cites superseded policy ['remote_work_v1']"]}
    <- "Employees could work remotely for a maximum of 2 days per week" [remote_work_v1]
{"valid": true, "issues": []}
    <- "Employees may work remotely for a maximum of 3 days per week" [remote_work_v2]
```

In the evaluation, the critic approved 8/8 answers on the cold run, with 1 retries needed.

Task 2.4: Tests and docs

PyTest: 22 passed in 11.27s (offline: a deterministic stand-in model emits real tool calls, so no key is needed):

- tests/test_tools.py: corpus fields, typed search hits, k clamp, full text, typed PolicyNotFoundError, soft tool error, calculator safe division and code rejection
- tests/test_agent_acceptance.py: doc_id + verbatim quote, bad-id recovery, ≥80% acceptance on the canned set, critic rejections, memory stores only verified facts, warm run uses fewer steps, no write after a failed critic
- tests/test_graph_budget.py: node order, budget stop, exactly one retry, retry skipped when the budget is too small, agent max_iterations enforced

Evaluation table (warm run) - pass rate 100%

id	kind	status	latency_s	agent_steps	graph_steps	has_docid	has_quote	cited	critic_valid	memory_hit	PASS
E1	factual	ok	3.891	1	6	True	True	remote_work_v2	True	True	True
E2	factual	ok	2.881	1	6	True	True	leave_v3	True	True	True
E3	factual	ok	2.83	1	6	True	True	travel_v4	True	True	True
E4	factual	ok	2.357	1	6	True	True	security_v1	True	True	True
E5	recovery	ok	4.0	2	6	True	True	remote_work_v2	True	True	True
E6	multi-tool	ok	2.817	2	6	True	True	travel_v4	True	True	True
E7	factual	ok	2.256	1	6	True	True	security_v1	True	True	True
E8	boundary	ok	2.027	1	6	False	False	-	True	False	True

Final evaluation table, printed by eval.py (gpt-4o-mini)

- **Acceptance:** cold 100% (8/8), warm 100% (8/8). Target ≥ 80%: met.
- **Docs and run scripts:** README.md (setup, run, design notes, limitations); app.py CLI (ask, eval --cold, eval, reset-memory, --report for a JSON run report, --sqlite); run_all.sh builds everything.
- **Screenshots in results/:** 02 cold run, 03 warm run (memory hits), 05 error recovery, 06 budget/retry, 07 pytest, 08 eval table.
- **Optional extensions done:** SQLite checkpointing (graph.sqlite_checkpointer, every node state persisted), a JSON run report via the CLI flag, and an optional LLM critic.
- **Limitations:** a tiny synthetic corpus; the deterministic critic checks numbers and quotes but not paraphrased claims; similarity thresholds were tuned on this corpus.